

There are **100** points possible.

Honor Pledge: I affirm that I will not give or receive any unauthorized help on this exam, and that all work will be my own.

Indicate agreement by typing or signing your name: _____

1. (14 pts) Let G be the grammar defined by the following rules (the start symbol is S):

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow b$

- a) Is the string ***aaaab*** a member of the language defined by G ? If so, show a derivation. If not, briefly explain why not.

- b) Is this grammar ambiguous? *Justify your answer.*

2. (6 pts) In mathematical expressions, parentheses can be arbitrarily added to sub-expressions. For example, all of the following expressions are equal:

$(7 + 5)$ $((7 + 5))$ $((7) + (5))$ $7 + 5$

In Racket, however, the following two expressions are *not* equal:

$(+ 7 5)$ $((+ 7 5))$

Which expression (the first or the second) would result in an error **and why?**

3. (20 pts) For each of the following Racket forms, show the resulting list structure in **boxes-and-arrows** representation.

a) `(cons 1 (cons 2 (cons 3 (cons 4 '()) )))`

b) `(cons 1 (cons 2 3))`

c) `(cons 1 (cons (cons 2 (cons 3 '())) (cons 4 (cons 5 '()))))`

d) `(append (cons 1 (cons 2 '())) (list (cons 3 4)) (list 5 6 7))`

4. (6 pts) Given the following Racket form:

```
(+ (* (/ (string->number "7") 2) (- bar foo)) 9)
```

- a) If this form were evaluated by Racket, what token would be evaluated first? You may assume that arguments are evaluated in **left-to-right** order. (Tokens include +, *, /, string->number, "7", 2, -, bar, foo, and 9)
- b) Which token would be evaluated next (second)?
- c) Which token would be evaluated third?

5. (10 pts) Write the Racket function `div-by` that accepts a single numeric argument and returns a list of all numbers less than 15 that the argument is divisible by.

(add to Racket file) your code for the function `div-by`

6. (20 pts) Consider the problem `count-evens`, defined as follows: Input: a list of integers. Output: the number of even integers in the list. For example, `(count-evens '(1 4 3 7 8))` should return the value 2 because the list contains two even numbers.

- a) Describe a recursive strategy for solving this problem in Racket *without using any helper functions or parameters other than the input list*. Be sure to describe how you would break the problem down into sub-problems, and how each sub-problem would be solved.

(add to Racket file) your code for function `count-evens`

- b) Now suppose that the input list can be nested. For example, you can call `(count-evens '(1 (2 3) (4 (5 6))))`. How would you modify your strategy from part-a to provide a correct algorithm for nested lists?

(add to Racket file) your modified code for `count-evens`

7. (8 pts) What is the difference between tail-call and non-tail-call recursion? Which one is more efficient in Racket, and why?

8. (10 pts) Consider the following Racket function:

```
(define sumsquares
  (lambda (lst)
    (if (null? lst)
        0
        (+ (* (car lst) (car lst)) (sumsquares (cdr lst)))))
```

Is this a tail-call? If so, explain why. If not, re-write the function so that it is a tail-call (add to Racket file).

9. (6 pts) a) What are Racket structures. Write some of the features of the Racket structures.

- b) Write a program that creates a structure “car” that has data members: type, model, color, num-doors, top-speed. Create three objects for three car makes, for example (ford, honda, ...). The data member type is a truck or sedan. Put values of your choice for the rest of the data members. The program then accesses the last three data members of each object to make changes (add to Racket file).